

# forml-core

FORML is a simple, human-readable DSL to describe formal properties (robustness, fairness, stability...) for ML models. It includes a parser, AST-to-Python transformer, examples, and a modular base to connect with formal verification tools like ERAN or Z3. # FORML – A Domain-Specific Language for Formal Verification in Machine Learning

**FORML** is a Domain-Specific Language designed to express *formal properties* that a machine learning model must satisfy. It bridges the gap between human-readable requirements and formal verification tools like **ERAN**, **Z3**, or others, enabling interpretable and auditable specification of constraints in ML pipelines.

- Built for clarity
- Focused on robustness, fairness, and logical consistency
- Compatible with formal verification backends

## Motivation

Traditional ML pipelines lack built-in support for specifying and verifying **model-level properties**, such as:

- Is my model robust to noise?*
- Does it treat sensitive attributes fairly?*
- Are the outputs bounded under input perturbation?*

**FORML** provides a simple, declarative syntax to define such properties — making **formal verification** accessible to non-experts and easily integrable into CI pipelines, research experiments, or audits.

## Project Structure (Overview)

FORML is organized as a modular Python project:

- grammar/ — Grammar definitions (EBNF, Lark)
- parser/ — Parser logic using Lark
- ast/ — Internal Python representation of properties
- examples/ — Sample .forml files
- tests/ — Tests
- README.md — DSL overview and usage
- ARCHITECTURE.md — Full file tree and detailed documentation

## Key Concepts

- A **FORML property** is a logical constraint defined in a user-friendly syntax.
- Each property can target:
  - Model behavior globally or at a specific instance.
  - Specific feature-based subgroups.
- Backends like **ERAN** or **Z3** are specified with using.

## Syntax Overview

### Basic Structure

```
[PROPERTY_TYPE]:  
for x with [DOMAIN] [at INSTANCE] := [ASSERTION] [using ABSTRACTION]
```

- PROPERTY\_TYPE:
  - ROBUSTNESS,
  - FAIRNESS,
  - LOGIC,
  - BOUND?
  - MONOTONICITY
  - STABILITY
- DOMAIN: defines the context of the test (e.g. feature range, noise, hyperball)
- ASSERTION: the constraint to be verified
- ABSTRACTION: optional — which backend method/parameters to use

## FORML v1 — Propriétés : Global vs Ciblé (Instance x₀)

| Property   | Global Syntax                                                                  | Global Semantics                                                    | Local Syntax (focus on x₀)                                                     | Local Semantics                                                             | Support Tool     | .forml Example                                                         |
|------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------|------------------------------------------------------------------------|
| ROBUSTNESS | ROBUSTNESS: for x in hyperball(L2, 0.01) -> CLASSIFICATION.EQUAL()             | All inputs must yield the same class under small L2 perturbations   | ROBUSTNESS: at x0 in hyperball(L2, 0.01) -> CLASSIFICATION.EQUAL()             | For a specific input x₀, its perturbed neighbors must yield same prediction | ERAN, ZONOTOPE   | ROBUSTNESS: at x0 in hyperball(L2, 0.01) -> CLASSIFICATION.EQUAL()     |
|            |                                                                                |                                                                     |                                                                                | Changing x₀'s sensitive attribute should not change the output beyond δ     |                  |                                                                        |
| FAIRNESS   | FAIRNESS: for x with gender("male", "female") -> PREDICTION.EQUITY(delta=0.05) | Across all subgroups, the prediction must remain within δ tolerance | FAIRNESS: at x0 with gender("male", "female") -> PREDICTION.EQUITY(delta=0.05) |                                                                             | Z3, custom rules | FAIRNESS: for x with gender("M", "F") -> PREDICTION.EQUITY(delta=0.05) |
|            |                                                                                | All slightly perturbed inputs must                                  |                                                                                |                                                                             |                  |                                                                        |
|            | BOUND: for x in                                                                |                                                                     | BOUND: at x0 in                                                                | Around x₀, no                                                               |                  | BOUND: at x0 in                                                        |

|                     |                                                                                           |                                                                                  |                                                                                           |                                                                                                    |                |                                                                                          |
|---------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|----------------|------------------------------------------------------------------------------------------|
| <b>BOUND</b>        | hyperball(L_inf, 0.01) -><br>PREDICTION.BETWEEN(0.0, 1.0)                                 | have<br>predictions<br>in a valid<br>range                                       | hyperball(L_inf, 0.01) -><br>PREDICTION.BETWEEN(0.0, 1.0)                                 | prediction<br>should<br>exceed limits                                                              | ERAN, BOX      | hyperball(L_inf, 0.01) -><br>PREDICTION.BETWEEN(0.0, 1                                   |
| <b>LOGIC</b>        | LOGIC: x.age > 18 -><br>ACCESS_GRANTED = True                                             | A logical rule<br>must hold for<br>any instance<br>satisfying a<br>condition     | (N/A)                                                                                     | (N/A)                                                                                              | Z3             | LOGIC: x.income > 10000 -<br>CREDIT_GRANTED = True                                       |
| <b>MONOTONICITY</b> | MONOTONICITY: for x in income<br>INCREASING -><br>PREDICTION.INCREASING                   | Increasing a<br>variable<br>should not<br>decrease the<br>prediction<br>globally | MONOTONICITY: at x0 in income<br>INCREASING -><br>PREDICTION.INCREASING<br>(optional)     | Locally<br>around x <sub>0</sub> ,<br>increasing a<br>variable must<br>not reduce<br>the output    | Custom /<br>Z3 | MONOTONICITY: for x in ag<br>INCREASING -><br>PREDICTION.INCREASING                      |
| <b>STABILITY</b>    | STABILITY: for x ~ x' with<br>distance(name, levenshtein,<br>1) -> CLASSIFICATION.EQUAL() | All close<br>symbolic<br>variants<br>should be<br>predicted the<br>same way      | STABILITY: at x0 ~ x' with<br>distance(name, levenshtein,<br>1) -> CLASSIFICATION.EQUAL() | Typos and<br>symbolic<br>perturbations<br>near x <sub>0</sub><br>should not<br>affect the<br>class | Custom /<br>Z3 | STABILITY: for x ~ x' wit<br>distance(city, levenshtein,<br>1) -> CLASSIFICATION.EQUAL() |

In FORML, every property can be expressed in a global form (for x ...) or a local form (at x0 ...). These two forms differ in purpose:

- The global form audits the model on the whole dataset.
- The local form is used to analyze model behavior around a specific instance, useful for debugging or interactive exploration.

Additionally, two types of constraint modifiers exist: - in: defines geometric neighborhoods (based on distance or norm) - with: expresses filters or symbolic constraints (on attribute values or semantic distances)

Example interpretation: - for x in hyperball(...): verify the model is invariant to local perturbations. - for x with gender(...): verify the model treats sensitive groups similarly. - at x0 with distance(...): check stability under symbolic variation around x<sub>0</sub>.

## File Structure and Declarations

A .forml file may contain:

- One or several property definitions
- Optional variable declarations (reusable across properties)

### Property Separator

Use --- to delimit variable declarations and sections if needed.

### Variable Declaration (Optional)

```
---
DELTA := 0.1
TARGET := sample_7
LABEL := "car"
---
```

Declared variables can be reused inside properties.

### Comments

Lines starting with # or // are treated as comments and ignored during parsing.

```
# This property ensures robustness under L_inf ball
ROBUSTNESS: for x with hyperball(L_inf, DELTA) at TARGET := PREDICTION.EQUAL()
```

### Abstractions (ERAN, etc.)

Declare the abstraction method to be used for the backend formal verifier with the using keyword:

```
using DeepPoly
using Zonotope(epsilon=0.1)
```

A list of supported abstractions and their parameters will be maintained in the documentation.

Example .forml File

```
---
DELTA := 0.05
SENSITIVE := gender("male", "female")
---

# Robustness near input_42
ROBUSTNESS:
for x with hyperball(L_inf, DELTA) at input_42 := CLASSIFICATION.EQUAL() using DeepPoly

# Fairness by gender
FAIRNESS:
for x with SENSITIVE := PREDICTION.EQUITY

# Logical constraint
LOGIC:
for x with hyperball(L_2, 0.01) := A and not B -> C
```

### How it Works (Under the Hood)

1. The FORML parser reads .forml files and converts them into an Abstract Syntax Tree (AST).
2. A Transformer converts this AST into Python objects representing each property.
3. The formal\_ml library:

- Matches each property with its compatible backend (e.g., ERAN, Z3)
- Executes the verification based on parameters and abstraction
- Returns structured, interpretable results

## Planned Ecosystem

| Component     | Description                                             |
|---------------|---------------------------------------------------------|
| FORML         | The DSL and its parser (based on Lark)                  |
| formal_ml     | Python interface to connect FORML with formal tools     |
| FM4ML         | Tutorial for introducing formal methods in ML via FORML |
| VSCode plugin | Syntax highlighting, autocomplete, and property support |

## License

Project licensed under MIT. See LICENSE.md for details.

## Author

Developed by KintanaOne (Alias of a data scientist & formal methods enthusiast who believes that ML should be tested as rigorously as software.)

“If AI is to be trusted, we must first speak to it in terms it can be tested against.”

## Contribute

- Contributions, suggestions, or improvements are welcome!
- Open issues to report bugs or suggest features
- Submit pull requests to improve the parser, DSL, or examples
- Star the repository if you find the idea promising

Let's bring formal verification closer to real-world ML together